

# BASE DE DATOS I

## Unidad 4: **Base de Datos Relacionales**

## Arquitectura de los sistemas de bases de datos

Una de las arquitecturas más utilizadas por los sistemas de bases de datos es la arquitectura ANSI/SPARC, que presenta 3 niveles conocidos como *interno*, *conceptual* y *externo*.

- El nivel externo (también conocido como lógico de usuario) es el más próximo a los usuarios; es decir, el que tiene que ver con la forma en que los usuarios individuales ven los datos.
- El nivel conceptual (también conocido como el lógico, sin calificar) es un nivel de interacción entre los otros dos.
- El nivel interno (también conocido como físico) es el que está más cerca del almacenamiento físico; es decir, es el que tiene que ver con la forma en que los datos están almacenados físicamente.

El nivel conceptual, es único. Si bien pueden existir muchas vistas del nivel externo, el nivel conceptual es único, es el modelo lógico de cómo se organizan y relacionan los datos dentro de la base de datos.

Por ejemplo:

Se tiene una base de datos que guarda datos de empleados de una organización.

En el nivel interno de la base de datos los empleados están representados por un registro de 20 bytes de longitud. Este registro posee 4 campos de la siguiente manera: los primeros 6 bytes corresponden al código de empleado, los próximos 4 bytes pertenecen al código de departamento, mientras que los últimos 10 bytes se corresponden con el salario del empleado.

En el nivel conceptual, la base de datos contiene información relacionada a una entidad llamada *Empleado* donde cada empleado posee un código (6 caracteres), un departamento (4 caracteres) y un salario (5 dígitos decimales).

Por último en el nivel externo existen 2 vistas una para usuarios de tesorería que pueden ver todos los datos de un empleado y otra vista para usuarios del departamento de RRHH que sólo pueden ver el código de empleado y departamento al que pertenecen.

### Nivel externo

Este nivel es el del usuario que bien puede ser un usuario final, un desarrollador de aplicaciones o un DBA (database administrator).



Para un usuario final, este nivel expone un lenguaje de consulta o bien una aplicación de finalidad específica que permite crear una consulta a través de elementos gráficos.

Para un usuario desarrollador, este nivel tendrá un lenguaje de programación convencional (Java, C++, PHP, etc.) con interfaces para interactuar con la base de datos. En principio estas interfaces serán una combinación de dos lenguajes ya mencionados que son DDL y DML.

- DDL consiste en aquellas construcciones declarativas necesarias para declarar objetos de base de datos (Tablas, campos, índices, restricciones, vistas, funciones).
- DML consiste en aquellas instrucciones ejecutables que transfieren información hacia y desde la base de datos (consultas, altas, bajas y modificaciones de datos)

Volviendo a la arquitectura, la vista externa o nivel externo es para un usuario final la base de datos propiamente dicha.

Cada vista externa, está definida mediante un **esquema conceptual** que consiste básicamente en definiciones de cada uno de los objetos de la base de datos.

## Nivel conceptual

La vista conceptual está definida por medio del esquema conceptual, el cual comprende definiciones de cada uno de los diversos tipos de registros conceptuales.

Para definir el esquema conceptual se utiliza el lenguaje de definición de datos DDL. Desde un punto de vista teórico y estricto, el esquema conceptual, no debe tener ninguna referencia sobre la representación física de los campos, formas de almacenamiento, restricciones de seguridad o integridad.

En resumen, el esquema conceptual es una forma de describir qué datos se almacenan dentro de la base de datos y sus relaciones

## Nivel interno

Este nivel es una representación de bajo nivel de toda la base de datos. La vista interna, está por delante del nivel físico, ya que no tiene que ver con cómo se definen las páginas de memoria o el tamaño de los cilindros de disco, etc.

La vista interna, sólo define los diversos tipos de registros, índices, etc.

## El modelo relacional

A modo de introducción, podemos decir que el modelo de datos entidad-relación (E-R) percibe el mundo real como un conjunto de objetos llamados entidades y de relaciones entre dichos objetos.

Es un sistema donde el usuario percibe la información como tablas, su aspecto estructural; estas tablas satisfacen ciertas restricciones de integridad, su aspecto de integridad; proporciona operadores para manipular los datos, son operadores que operan sobre tablas y su resultado deriva en tablas, aspecto de manipulación.

### Conceptos básicos

**Entidad:** es un objeto en el mundo real que es distinguible de todos los demás objetos. Por ejemplo una persona, una cuenta bancaria, un libro. Una entidad posee un conjunto de propiedades o atributos que la describen y que también permiten identificarla de forma unívoca.

Cada uno de los atributos de una entidad posee un conjunto posible de valores que puede tomar que es llamado **dominio**. Por ejemplo, el dominio del atributo *edad* de una entidad *Persona* podría ser los valores enteros entre 0 y 130.

Dentro de una base de datos relacional, una entidad está representada por un registro y sus atributos por campos. El conjunto de entidades del mismo tipo se agrupan dentro de una tabla.

**Registro:** el término registro, **fila** o **tupla** todos significan lo mismo. Son términos usados para describir una entidad, un conjunto de atributos, dentro de una tabla.

**Campo:** el término campo, **columna** o **atributo** todos significan lo mismo. Son términos usados para describir un campo en una tabla. Los campos se repiten en cada registro (entidad), pero no necesariamente sus valores van a repetirse entre una entidad y otra.

**Tabla:** es la representación de un conjunto de entidades del mismo tipo dentro de una base de datos relacional. Es una estructura lógica. Las tablas son una abstracción de cómo se almacenan los datos físicamente.

Como mencionamos anteriormente, es una estructura de datos compuesta por campos y registros donde todos los registros poseen los mismo campos (no valores), es decir comparten la misma estructura.

En la introducción comentamos que los operadores actúan sobre tablas y su resultado es otra tabla, esto es a nivel conceptual, no significa que el resultado debe materializarse en una tabla en la base de datos. Es importante destacar de este aspecto que como todos los operadores operan sobre una tabla y su resultado es otra tabla, entonces podemos decir que el resultado de una operación puede ser la entrada de otra y esto nos permite anidar operaciones

| Entidad / Registro  |            |            |               |
|---------------------|------------|------------|---------------|
| Atributo /<br>campo | ISBN       | AUTOR      | TITULO        |
|                     | 1585670081 | Juan Perez | Base de datos |
|                     | 345333926  | John Doe   | C++ explained |

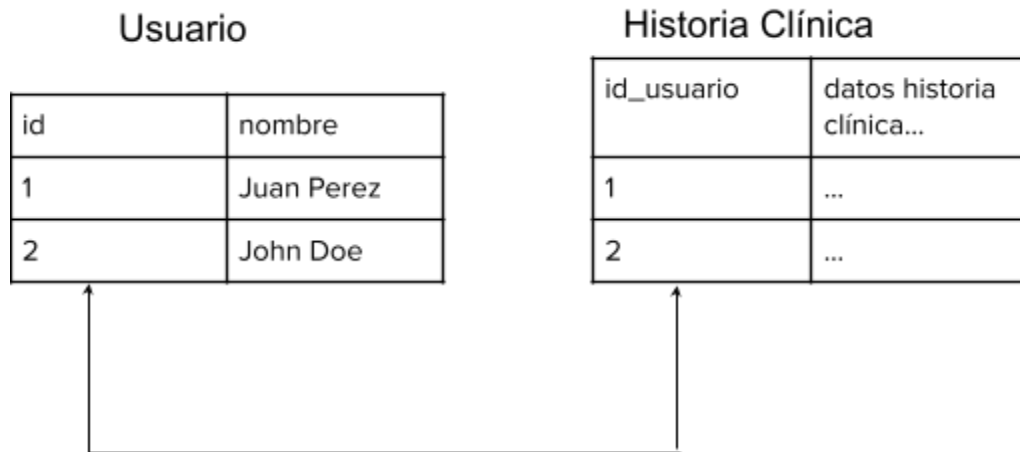
## Relaciones

Una relación es una asociación entre diferentes entidades. Por ejemplo, se podría asociar una entidad *libro* con una entidad *autor*.

La cantidad de tablas que intervienen en una relación es el grado de la misma, entonces si en una relación intervienen 2 tablas, será una relación binaria, si intervienen 3 tablas será ternaria y así sucesivamente. Lo más común es que el grado sea binario.

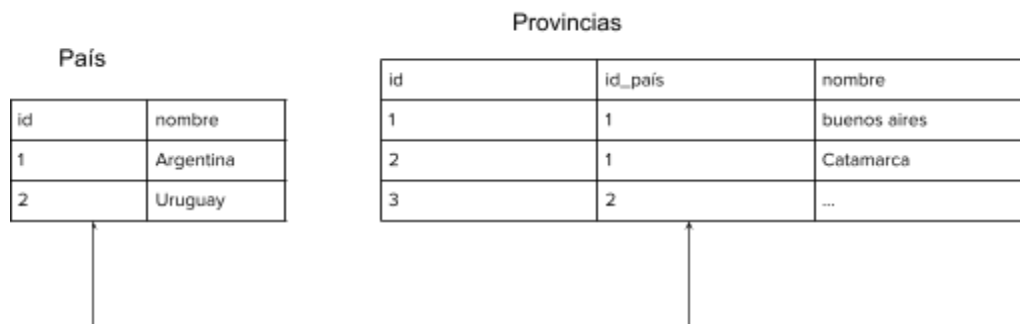
Además del grado, las relaciones poseen otro atributo que es la cardinalidad. Este atributo expresa el número de entidades a las que otra entidad puede estar asociada. Los tipos de cardinalidad más comunes son:

- Uno a Uno (1 - 1): una entidad en A se asocia con a lo sumo una entidad en B y viceversa. Por ejemplo, una persona y su historia clínica.



Las relaciones 1 a 1 son poco comunes, son utilizadas para eliminar repetidos valores NULL en campos.

- Uno a muchos (1 - N): una entidad en A se asocia con uno o más en la entidad B pero una entidad en B se asocia sólo con 1 entidad en A. Por ejemplo un país y sus provincias.



Las relaciones 1 - N son las más comunes en las bases de datos relacionales.

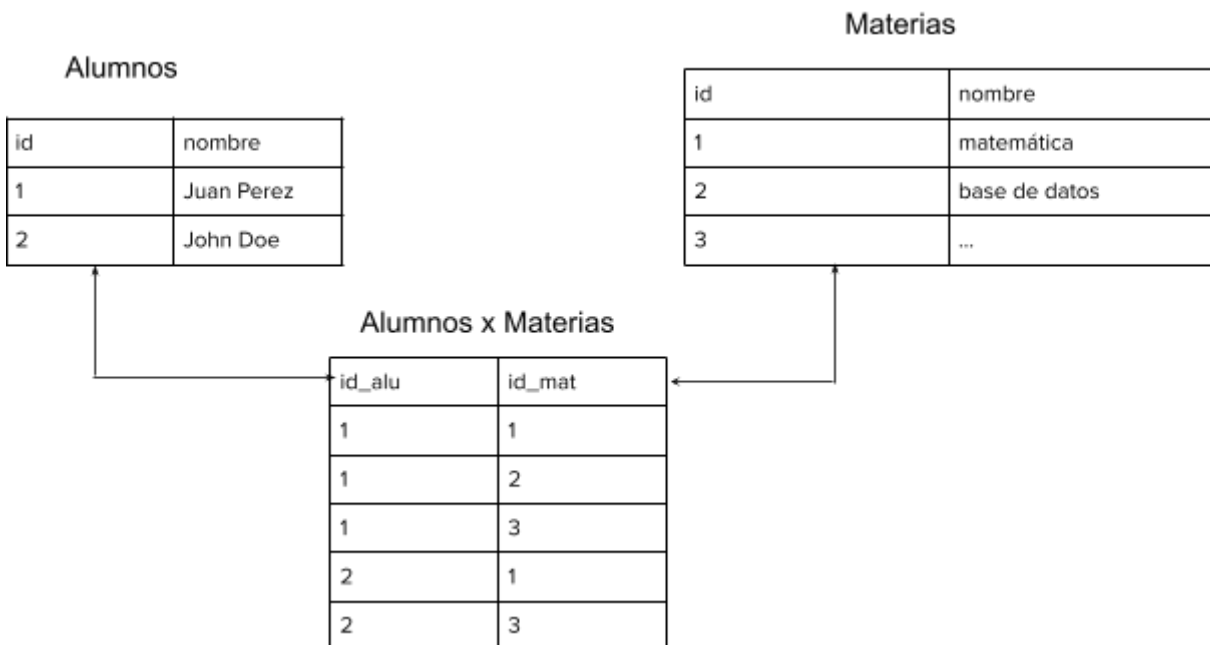
- Muchos a muchos (N - M): una entidad en A se asocia con uno o más en la entidad B y una entidad en B se asocia con uno o más entidades en A. Por ejemplo alumnos que pueden inscribirse a muchas materias.

Si bien este tipo de relaciones se puede resolver en una única tabla, posee el problema de la redundancia de datos y aún más importante se hace costoso obtener un único alumno o una única materia.

## Datos

| nombre     | materia       |
|------------|---------------|
| Juan Perez | matemática    |
| Juan Perez | base de datos |
| John Doe   | base de datos |
| John Doe   | ...           |

Para resolver estos problemas, se debe crear una tabla intermedia y 2 relaciones 1 - N.



## Restricciones sobre las relaciones.

Las relaciones entre entidades pueden ser obligatorias o únicas (o no repetibles) y combinaciones de éstas.

**Obligatorias:** implica que el atributo que referencia una entidad en la tabla relacionada debe referenciar de forma obligatoria una entidad en la tabla relacionada. En contrapartida, una relación no obligatoria permite que el atributo de la entidad quede vacío.

**Únicas:** implica que el atributo que referencia una entidad en la tabla relacionada no puede ser repetido entre entidades de la misma tabla.

**Obligatorias y Únicas:** implica que el atributo que referencia una entidad en la tabla relacionada no puede repetirse y tampoco puede quedar vacío.

## Claves

En una base de datos relacional, la palabra *clave* (*key*) es un término usado para denotar campos que pueden vincular tablas, es decir, participar de una relación. Una clave puede estar conformada por uno o más campos.

Una clave es además un índice (ver [Índices](#)), porque los valores de ese campo son copiados a una estructura de búsqueda más eficiente.

Existen 3 tipos de claves: Primaria, Única y Foránea.

### Clave Primaria (Primary Key)

Una clave primaria es utilizada para identificar de forma unívoca un registro dentro de una tabla.

Al definir un campo, o conjunto de campos, como clave primaria lo que estamos haciendo es establecer una restricción sobre ese o esos campos donde los mismos no pueden repetirse entre registros ni quedar vacíos, en otras palabras, los campos en cuestión son únicos y obligatorios.

Por lo general y por cuestiones de performance (rendimiento) en las consultas, los campos que conforman una clave primaria serán campos de tipo entero (integer).

A la hora de elegir el o los campos que conformarán la clave primaria, debemos pensar en aquellos campos que identifican de forma unívoca una entidad. Por ejemplo en el caso de *personas* podríamos utilizar el DNI como clave primaria.

Sólo puede definirse una única clave primaria por tabla.

### Clave Única (Unique Key)

Al igual que con las claves primarias, una clave única establece una restricción de unicidad sobre el o los campos involucrados. Esto significa que los valores de dichos campos no pueden repetirse entre los registros de la tabla, deben ser únicos.

Distinto a lo que ocurre con las claves primarias, se puede definir tantas claves únicas como se necesiten.

Los campos definidos como *clave única* no son utilizados en relaciones entre tablas, a menos que dicho campo posea a su vez una restricción de obligatoriedad (campo no nulo).



Por ejemplo, en el caso de *usuarios* de un sistema se podría definir una clave única sobre el campo *email*.

### Clave Foránea (Foreign Key)

Una *clave foránea* es un campo cuyos valores serán las copias del campo definido como *clave primaria* en la tabla “padre” de la relación. Una *clave foránea* define la referencia para cada registro dentro de la tabla “hijo” de la relación. hacia los registros de la tabla “padre” de la relación.

Por ejemplo en el caso de tabla *Países y Provincias*, en la 2da tabla vamos a tener una columna *pais\_id* que será una clave foránea a los registros de la tabla *País*.

Por definición un campo sobre el que se define una clave foránea puede ser vacío o en caso de tener un valor, el mismo debe ser uno de los valores del campo referenciado en la tabla padre.

Al igual que las claves únicas, se pueden definir todas las claves foráneas que se necesiten.

Un campo de una tabla “hijo” sólo puede participar de una clave foránea.

### Integridad referencial

Como su nombre lo indica, la integridad referencial, asegura la integridad de las relaciones definidas por claves primarias y foráneas.

Este mecanismo, asegura que si sobre un campo se define una clave primaria, los valores del mismo no se repitan entre los registros de la tabla en cuestión y que no sean vacíos (nulos). En el caso de una relación entre tablas verifica que los valores cargados en un campo sobre el que se definió una clave foránea, existan en el campo referenciado (que por lo general será una clave primaria):

También verifica que si se intenta modificar los datos de un campo que participa en una relación (o eliminar el registro), no queden registros huérfanos en tablas “hijo”.

## Índices

### Qué es un índice

Un índice, es una estructura que almacena datos copiados del campo de la tabla sobre el cual se definió el índice. Su función, al igual que el índice de un libro, es permitir encontrar algo de forma más eficiente, en un libro será un capítulo del libro mientras que en tabla será un registro de la misma.

La forma de almacenar los índices varía de DBMS a DBMS, lo importante es que todos establecen la separación en el nivel físico.

### Cómo funciona un índice

Cuando realizamos una consulta a la base de datos, el *Procesador de Consultas* (el encargado de generar los planes de consultas) define si debe recorrer la tabla completa (escaneo secuencial) o si utilizará alguno de los índices definidos para obtener los registros solicitados de forma más rápida.

No siempre el acceso a través de los índices es el más rápido, por ejemplo si la tabla posee pocos registros siempre será más rápido un acceso secuencial que un acceso a través de alguno de los índices definidos.

Otra cuestión a tener en cuenta es que cada actualización a una tabla (crear, modificar o eliminar un registro) impactará en los índices que la misma posea, por ende una tabla con muchos índices se traducirá en una baja en el rendimiento en las operaciones de alta, baja o modificación de registros.

### Tipos de índices

Existen distintos tipos de índices, a continuación mencionamos algunos de los más comunes:

#### **B-Tree**

- Ideal sobre columnas que posean valores únicos.
- Puede proveer los datos ordenados
- Mejor performance en temas de concurrencia

#### **Generalized Search Tree (GIST)**

- Ideal para indexar datos complejos que no se “adaptan” a un índice b-tree
- Permite indexar datos geográficos, arrays, par clave/valor

## Generalized Inverted Index (GIN)

- Ideal para indexar valores compuestos
- Ideal para indexar columnas con muchos duplicados, ya que por cada clave puede tener punteros a varias filas. Esto también reduce el tamaño del índice
- Este índice guarda un conjunto de pares (clave, valor) donde el “valor” es un conjunto de IDs de filas donde ocurre esa clave
- Muy utilizado en valores de tipo: documento, JSON, arrays

## Otros

- Índices por expresiones (Expression Index)
  - Permite definir el índice a partir de una expresión, función, concatenación, etc.
  - Ej: lower(nombre)
- Índices parciales (Partial Index)
  - Permite definir un índice para un subconjunto de filas y no toda la tabla
  - Reduce el overhead en operaciones de insert/update
  - Reduce el tamaño del índice
  - Ej: CREATE INDEX state\_idx ON customer (state) WHERE state = 'AZ';

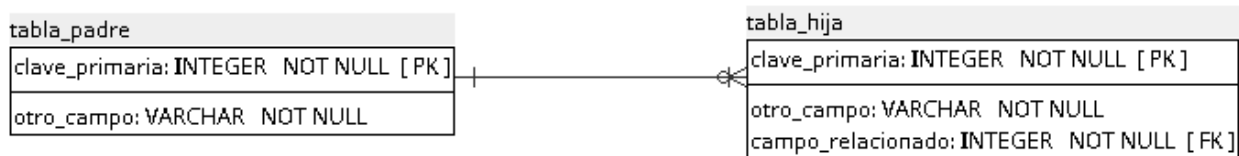
## Resumen

- Sólo B-tree, GIN y GIST soportan índices multicolumna
- B-tree ideal sobre columnas que posean valores únicos.
- GIN Ideal para indexar columnas con muchos duplicados
- GIST para todo lo demás

## Diagrama Entidad-Relación



Ejemplo de relación 1 - 1



Ejemplo de relación 1 - N

## SQL - DDL

El “sublenguaje” DDL (Data Definition Language) es el conjunto de sentencias que permite administrar las estructuras de la base de datos. Se compone de instrucciones que permiten crear, modificar y eliminar los objetos de una base de datos relacional como ser: tablas, campos, índices, vistas, etc.

### Creación de Tablas

Como mencionamos anteriormente una tabla es una estructura lógica, compuesta por columnas (campos) y filas (registros), donde cada campo posee un nombre y tipo de dato específico.

PostgreSQL nos presenta el comando **CREATE TABLE** que nos permite crear tablas.

La sintaxis básica de este comando sería:

```
CREATE TABLE <nombre_de_tabla> (  
  <nombre_de_campo> <tipo> <restricciones sobre el campo>,  
  <nombre_de_campo_2> <tipo> <restricciones sobre el campo>,  
  ...  
  <nombre_de_campo_N> <tipo> <restricciones sobre el campo>
```

```
<restricciones sobre la tabla>
);
```

Un ejemplo simple de este comando sería:

```
CREATE TABLE my_first_table (
    first_column text,
    second_column integer
);
```

## Campos y restricciones sobre campos

Cómo se vio en el ejemplo anterior, el comando ***CREATE TABLE*** también nos permite especificar los campos y sus restricciones.

La sintaxis básica para agregar un campo es:

```
<nombre_del_campo> <tipo_de_dato> <restricciones>,
```

Los tipos de datos más comunes son:

| Numéricos                                                                                                                                                                 | Texto                                                                                                  | Date                                                                       | Boolean                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------|
| <ul style="list-style-type: none"><li>• smallint</li><li>• smallserial</li><li>• integer</li><li>• serial</li><li>• bigint</li><li>• bigserial</li><li>• double</li></ul> | <ul style="list-style-type: none"><li>• text</li><li>• character</li><li>• character varying</li></ul> | <ul style="list-style-type: none"><li>• date</li><li>• timestamp</li></ul> | <ul style="list-style-type: none"><li>• boolean</li></ul> |

Las restricciones más comunes sobre los campos son:

- NOT NULL: especifica que el valor de dicho campo no puede ser null, es decir, el campo es obligatorio.
- UNIQUE: especifica que el valor de dicho campo no puede repetirse entre registros
- PRIMARY KEY: especifica que el valor de dicho campo es obligatorio y no puede repetirse entre registros

- CHECK: permite realizar validaciones sobre el campo
- REFERENCES: especifica que dicho campo es una clave foránea.

### Ejemplos

```
CREATE TABLE usuarios (  
    id bigserial PRIMARY KEY,  
    name character varying (30) NOT NULL,  
    surname character varying (30) NOT NULL,  
    email character varying (30) NOT NULL UNIQUE,  
    age smallint CHECK (age > 0),  
    type char DEFAULT ('C')  
);
```

```
CREATE TABLE telefonos (  
    id serial CONSTRAINT tel_pk PRIMARY KEY,  
    id_usuario bigint REFERENCES usuarios (id),  
    telefono_1 character varying (15),  
    telefono_2 character varying (15),  
    telefono_3 character varying (15)  
);
```

```
CREATE TABLE direcciones (  
    id serial,  
    id_usuario bigint,  
    direccion_1 character varying (50),  
    direccion_2 character varying (50),  
  
    CONSTRAINT dir_pk PRIMARY KEY(id),  
    CONSTRAINT usr_fk FOREIGN KEY (id_usuario) REFERENCES usuarios (id)  
);
```

Para una definición completa de este comando ver:

<https://www.postgresql.org/docs/current/static/sql-createtable.html>

## Modificación de la estructura de una tabla

En caso de que necesitemos modificar la estructura de una tabla ya creada, debemos utilizar el comando **ALTER TABLE**. Este comando nos permitirá renombrar la tabla y sus campos, cambiar el tipo de datos de los campos (siempre que no existan conflictos con los datos o relaciones), agregar o eliminar campos, etc.

Sintaxis básica:

```
alter table <nombre_de_tabla> <acción>;
```

Ejemplos:

Agregar una columna

```
alter table usuarios add column description character varying (50);
```

Eliminar una columna

```
alter table usuarios drop column description;
```

Renombrar una columna

```
alter table usuarios rename column description to comments;
```

Cambiar el tipo de dato de una columna

```
alter table usuarios alter column age type integer;
```

Agregar una restricción a un campo

```
alter table usuarios alter column comments SET NOT NULL;
```

Eliminar una restricción a un campo

```
alter table usuarios alter column comments DROP NOT NULL;
```

Para una definición completa de este comando ver:

<https://www.postgresql.org/docs/current/static/sql-altertable.html>

## Índices

Como se mencionó anteriormente, un índice es una estructura ordenada que facilita el acceso rápido (en la mayoría de los casos) a los datos de una tabla.

PostgreSQL soporta varios tipos de índices: B-tree, Hash, GiST, SP-GiST y GIN. Tener en cuenta que cada uno utiliza un algoritmo de búsqueda diferente.

Por defecto, el comando ***CREATE INDEX*** crea índices de tipo B-tree ya que es el que mejor se adecúa a la mayoría de los casos.

### Sintaxis básica

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ name ] ON table [ USING method ]( { column | ( expression ) }[ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )[WHERE predicate]
```

- Concurrently: permite crear el índice sin bloquear la tabla
- Method: uno de btree, hash, gin, gist, spgist, brin. Si no se especifica asume btree
- Expression: permite definir una expresión para crear el índice.
- Predicate: expresión para definir un índice parcial.

Siempre que se cree un índice sobre una tabla con datos y también periódicamente se debe ejecutar el comando:

```
VACUUM FULL ANALYZE [VERBOSE] <nombre_de_tabla>;
```

Este comando realiza un mantenimiento de los índices sobre la tabla y sobre sus datos.

### Eliminar una tabla

```
drop table <nombre_de_tabla>;
```

## SQL - DML

El “sublenguaje” DML (Data Manipulation Language) es el conjunto de sentencias que permite administrar las estructuras de la base de datos. Se compone de instrucciones que permiten crear, modificar y eliminar entidades en una base de datos relacional .

### Insertar datos

Para insertar datos en una tabla, contamos con el comando ***INSERT INTO***. Este comando permite insertar 1 registro en una tabla, especificando los campos afectados y sus valores.

#### Sintaxis básica

```
insert into <tabla> (<campo_1>, <campo_2>, ..., <campo_n>) values  
(<valor_campo_1>, <valor_campo_2>, ..., <valor_campo_n>);
```

Ejemplos:

```
insert into usuarios (name, surname, age) values ('Juan', 'Perez', 20);
```

Tener en cuenta que aquellos campos sobre los que se definió una secuencia (autonumerados) no deberían ser especificados a menos que se les quiera establecer un valor distinto al que asignaría la secuencia.

Los campos que tienen un valor por defecto en caso de que se especifiquen en la sentencia *insert* serán tenidos en cuenta y se insertarán con el valor especificado. Si no se especifican se insertarán con el valor por defecto definido.

Otra forma de realizar un insert es utilizando un query para especificar los valores.

```
insert into <tabla> (<campo_1>, <campo_2>, ..., <campo_n>)  
select campo_x, campo_y, ..., campo_z  
from <tabla>;
```

## Actualizar datos

Para actualizar datos en una tabla, contamos con el comando **UPDATE**. Este comando permite actualizar 1 o más registros en una tabla.

### Sintaxis básica

```
update <tabla> set <campo_1> = <valor_1>, <campo_2> = <valor_2>, ...,  
<campo_n> = <valor_n>  
[where <condición>];
```

Ejemplos:

```
update usuarios set description = 'hola mundo' where id < 10;
```

## Eliminar datos

Para eliminar datos en una tabla, contamos con el comando **DELETE**. Este comando permite eliminar 1 o más registros en una tabla.

### Sintaxis básica

```
delete from <tabla> [where <condición>];
```

Ejemplos:

```
delete from usuarios where id < 10;
```

## Consultar datos

Para consultar datos en una tabla, contamos con el comando **SELECT**. Este comando permite consultar 1 o más registros en una tabla o varias tablas.

### Sintaxis básica

```
select <campos| *> from <tabla>;
```

Ejemplos:

```
-- devuelve todos los campos de la tabla alumnos.  
select * from alumnos;  
  
-- devuelve legajo, apellido y nombre de la tabla alumnos  
select legajo, apellido, nombre from alumnos;
```

### Sintaxis completa

```
SELECT [ ALL | DISTINCT [ ON ( expression [, ...] ) ] ]  
[ * | expression [ [ AS ] output_name ] [, ...] ]  
[ FROM from_item [, ...] ]  
[ WHERE condition ]  
[ GROUP BY grouping_element [, ...] ]  
[ HAVING condition [, ...] ]  
[ { UNION | INTERSECT | EXCEPT } [ ALL | DISTINCT ] select ]  
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS { FIRST |  
LAST } ]  
[, ...] ]  
[ LIMIT { count | ALL } ]  
[ OFFSET start [ ROW | ROWS ] ]
```



**SELECT:** Palabra clave que indica que la sentencia de SQL que queremos ejecutar es de selección.

**ALL:** Indica que queremos seleccionar todos los valores. Es el valor por defecto y no suele especificarse casi nunca.

**DISTINCT:** Indica que queremos seleccionar sólo los valores distintos.

**FROM:** Indica la tabla (o tablas) desde la que queremos recuperar los datos. En el caso de que exista más de una tabla se denomina a la consulta "consulta combinada" o "join". En las consultas combinadas es necesario aplicar una condición de combinación a través de una cláusula WHERE.

**WHERE:** Especifica una condición que debe cumplirse para que los datos sean devueltos por la consulta. Admite los operadores lógicos AND y OR.

**GROUP BY:** Especifica la agrupación que se da a los datos. Se usa siempre en combinación con funciones agregadas.

**HAVING:** Especifica una condición que debe cumplirse para que los datos sean devueltos por la consulta. Su funcionamiento es similar al de WHERE pero aplicado al conjunto de resultados devueltos por la consulta. Debe aplicarse siempre junto a GROUP BY y la condición debe estar referida a los campos contenidos en ella.

**ORDER BY:** Presenta el resultado ordenado por las columnas indicadas. El orden puede expresarse con ASC (orden ascendente) y DESC (orden descendente). El valor predeterminado es ASC.

**LIMIT:** permite establecer la cantidad de registros a devolver

**OFFSET:** establece el número de registro a partir del cual se devuelven registros.

LIMIT y OFFSET son utilizados para armar paginado de datos.

## Funciones de agregado

Las funciones de agregado realizan un cálculo sobre un conjunto de valores y devuelven un solo valor.

Las funciones de agregado se suelen utilizar con la cláusula `GROUP BY` de la instrucción `SELECT`.

Todas las funciones de agregado son deterministas. Esto significa que las funciones de agregado devuelven el mismo valor cada vez que se las llama con un conjunto específico de valores de entrada.

Con las funciones de agregado de SQL, se pueden determinar varias estadísticas relacionadas con conjuntos de valores.

Las funciones de agregado son:

**avg:** permite obtener el promedio de valores de un campo

**count:** permite contar valores de un campo. Se puede utilizar en conjunción con la cláusula *distinct* para sólo contar los valores distintos sobre un campo

**min:** obtiene el mínimo valor de un campo

**max:** obtiene el máximo valor de un campo

**sum:** suma los valores de un campo

Ejemplos:

Dada la tabla *empleados*:

| Legajo | Nombre  | Apellido  | Departamento | Salario |
|--------|---------|-----------|--------------|---------|
| 123    | Juan    | Perez     | Sistemas     | 40000   |
| 456    | Ana     | Martinez  | Sistemas     | 45000   |
| 876    | Valeria | Rodriguez | Legales      | 50000   |



```
-- Obtener la cantidad de empleados de cada departamento
select departamento, count(*)
from empleados
group by departamento;
```

```
-- Obtener la cantidad de departamentos a partir de la tabla empleados
select count(distinct departamento)
from empleados;
```

```
-- Obtener el salario mínimo, el salario máximo y promedio de los salarios
mayores a 20000 por departamento
select departamento, min(salario), max(salario), avg(salario)
from empleados
group by departamento
having avg(salario) > 20000;
```



### **Bibliografía:**

- Diseño y Administración de Base de Datos Ing. Alejandro Mostovoi CUVL- 2018
- GavinPowell. Beginning Database Design. Indianapolis: Wiley Publishing, Inc, 2006. ISBN-13: 978-0-7645-7490-0.
- AbrahamSilberschatz; Henry F. Korth; S. Sudarshan. FUNDAMENTOS DE BASES DE DATOS. Cuarta Edición. España: McGRAW-HILL/INTERAMERICANA DE ESPAÑA, S. A. U, 2002.
- C.J.Date. Introducción A Los Sistemas De Bases De Datos. Séptima Edición. Méjico: Pearson Educación, 2001. ISBN: 968-444-419-2
- PostgreSQL Current Documentation. Disponible en: <https://www.postgresql.org/docs>